



BBF Review

가짜연구소 11기 러너 진승정



Key Ideas

SPR: Rainbow + self-supervised objective + augmentation

SR-SPR: SPR + replay ratio + model param resetting

BBF: SR-SPR w/ scaling

SPR (2021)

DATA-EFFICIENT REINFORCEMENT LEARNING WITH SELF-PREDICTIVE REPRESENTATIONS



Motivation

- Inefficient RL algorithms: 10-50 years, 45,000 years of experience
- DQN: Replay ratio = $1/4$
- Self supervised learning: Massive success in data efficiency in other areas

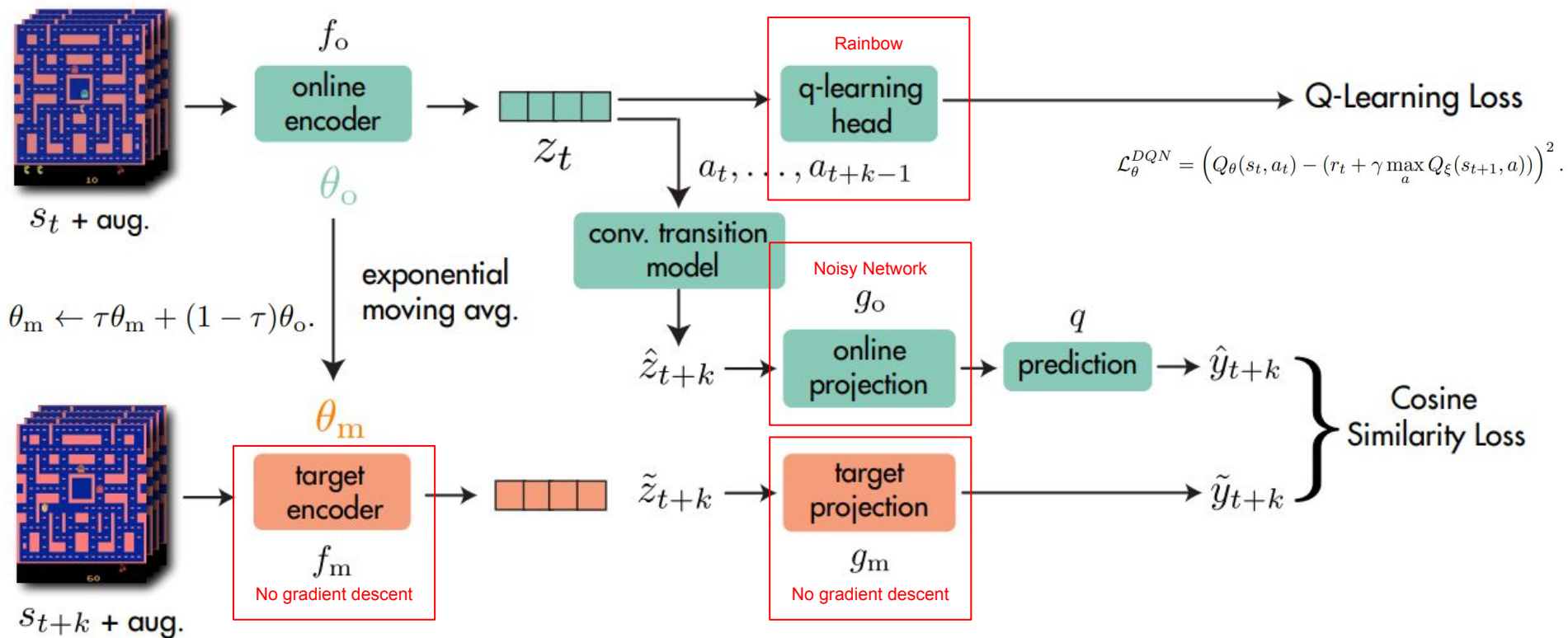
Self-supervision으로 data efficiency를 풀 수 있지 않을까?



Key Idea

"can learn more efficiently if we **augment reward maximization** with **self-supervised objectives** based on structure in its visual input and sequential interaction with the environment"

- Rainbow
- Self supervised: Predict future observations with temporal offset, K
- Latent space: Work latents, not pixel space
- Augmentation: Augmentation on pixels (channel jitter, random shift)





Augmentation

- Augmentation
 - Small random shift
 - color jitter
- No augmentation
 - 0.5 dropout in each layer of online and target encoders



Objective

- SPR Loss

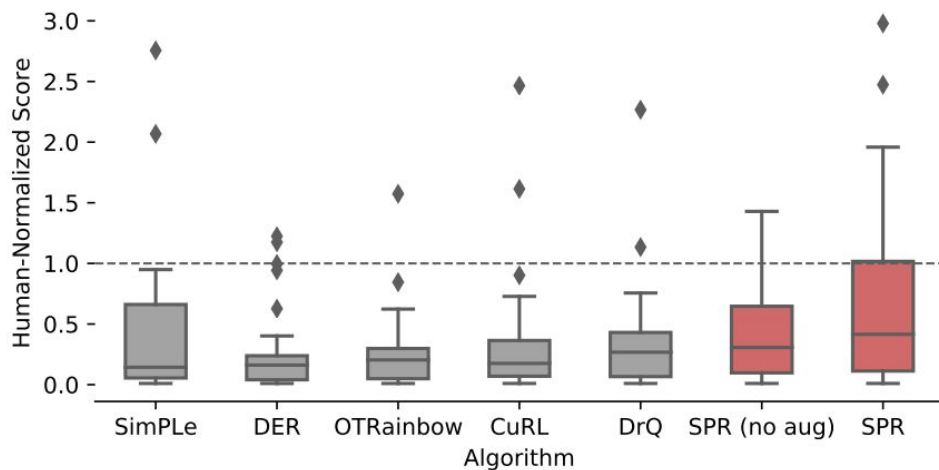
$$\mathcal{L}_{\theta}^{\text{SPR}}(s_{t:t+K}, a_{t:t+K}) = - \sum_{k=1}^K \left(\frac{\tilde{y}_{t+k}}{\|\tilde{y}_{t+k}\|_2} \right)^{\top} \left(\frac{\hat{y}_{t+k}}{\|\hat{y}_{t+k}\|_2} \right),$$

- Exponential Moving Average (EMA) update ($\tau = 0.99$ without augmentation, $\tau = 0$ with aug)

$$\theta_{\text{m}} \leftarrow \tau \theta_{\text{m}} + (1 - \tau) \theta_{\text{o}}.$$

Re:

Metric	Random	Human	SimPLE	DER	OTRainbow	CURL	DrQ	SPR (no Aug)	SPR
Mean Human-Norm'd	0.000	1.000	0.443	0.285	0.264	0.381	0.357	0.463	0.704
Median Human-Norm'd	0.000	1.000	0.144	0.161	0.204	0.175	0.268	0.307	0.415
Mean DQN@50M-Norm'd	0.000	23.382	0.232	0.239	0.197	0.325	0.171	0.336	0.510
Median DQN@50M-Norm'd	0.000	0.994	0.118	0.142	0.103	0.142	0.131	0.225	0.361
# Games Superhuman	0	N/A	2	2	1	2	2	5	7



SR (Scaled by Resetting) - SPR (May 2023)

SAMPLE-EFFICIENT REINFORCEMENT LEARNING BY BREAKING THE REPLAY RATIO BARRIER



Key Idea

- Scale replay ratio (number of updates of an agent's parameters for each environment interaction)
- reset plasticity: Reset the model partially or fully every n steps

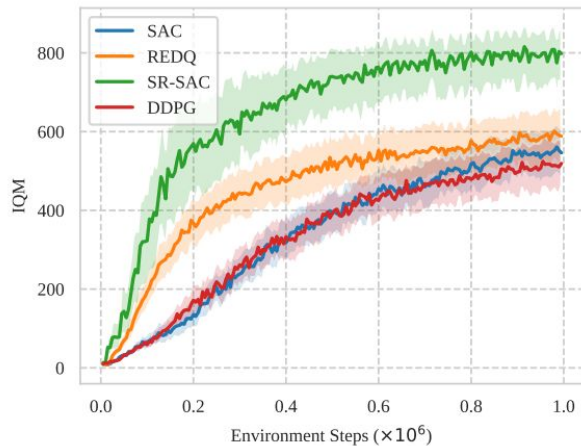
DL network의 catastrophic forgetting 등의 문제를 해결하면 기존 데이터를 더 활용할 수 있지 않을까?



Loss of Ability to Learn and Generalize in Neural Networks

- NN lose ability to learn and generalize during training
- Worse in when data distribution shifts
- Partial reset helps for continual learning

SR-SAC



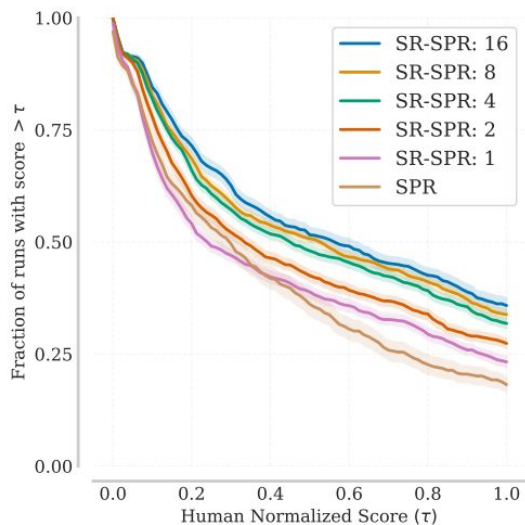
DMC15-500k			
Method	IQM	Median	Mean
SR-SAC	740 (642, 818)	667 (573, 742)	658 (589, 722)
REDQ	511 (440, 577)	493 (442, 544)	494 (452, 534)
SAC	391 (334, 448)	424 (376, 468)	424 (386, 461)
DDPG	392 (334, 445)	410 (364, 454)	408 (371, 442)

DMC15-1M			
Method	IQM	Median	Mean
SR-SAC	805 (726, 867)	729 (628, 790)	710 (643, 775)
REDQ	586 (514, 649)	546 (490, 596)	539 (498, 576)
SAC	535 (467, 597)	525 (471, 567)	519 (480, 557)
DDPG	514 (450, 572)	492 (440, 540)	489 (450, 526)

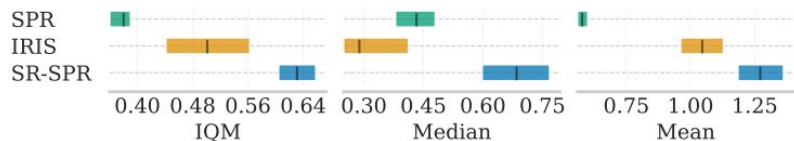
Figure 2 & Table 1: Performance of SR-SAC and of standard baselines on the DMC15 benchmark. (5 seeds for SR-SAC, 20 for all other algorithms, 95% bootstrapped C.I.).

SR-SPR

- Atari 100k Benchmark:



Atari 100k			
Method	IQM	Median	Mean
SR-SPR	0.632 (0.60, 0.66)	0.685 (0.60, 0.77)	1.272 (1.18, 1.37)
IRIS	0.501 (0.44, 0.56)	0.289 (0.25, 0.41)	1.046 (0.96, 1.13)
SPR	0.380 (0.36, 0.39)	0.433 (0.38, 0.48)	0.578 (0.56, 0.60)
DrQ(ϵ)	0.280 (0.27, 0.29)	0.304 (0.28, 0.33)	0.465 (0.46, 0.48)
DER	0.183 (0.18, 0.19)	0.191 (0.18, 0.21)	0.351 (0.34, 0.36)



Data / Compute Trade off

- Collect new samples vs. compute

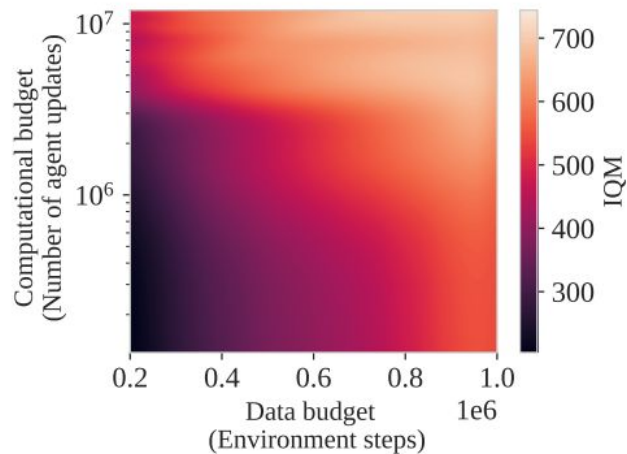


Figure 8: Performance of SR-SAC in DMC15 as a function of the number of interactions and of the number of agent updates, determined by the replay ratio.



Limitations

- Replay ratio scaling limit is not automatically identifiable
- Cannot go beyond limitations of given algorithm
- Size of data that can fit into replay buffer

BBF (Nov 2023)

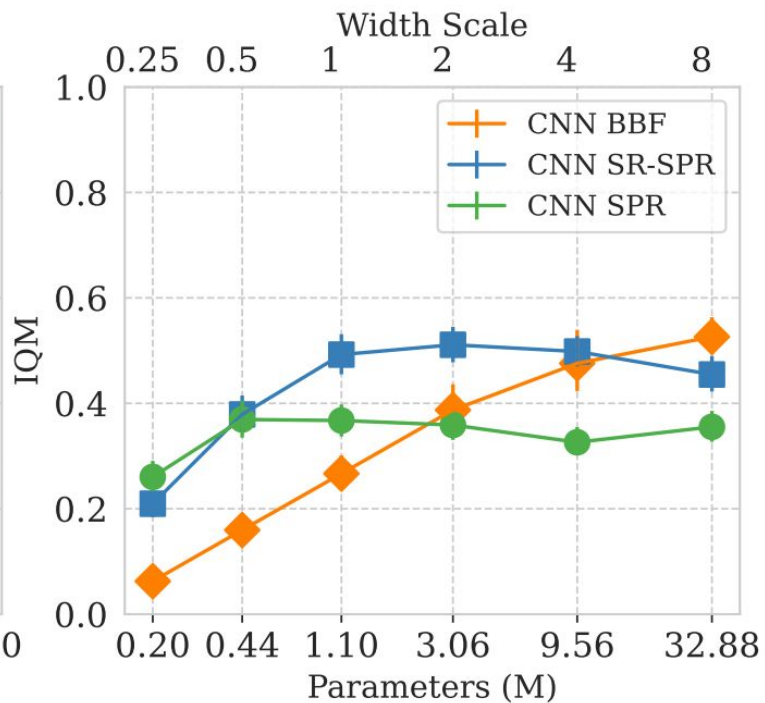
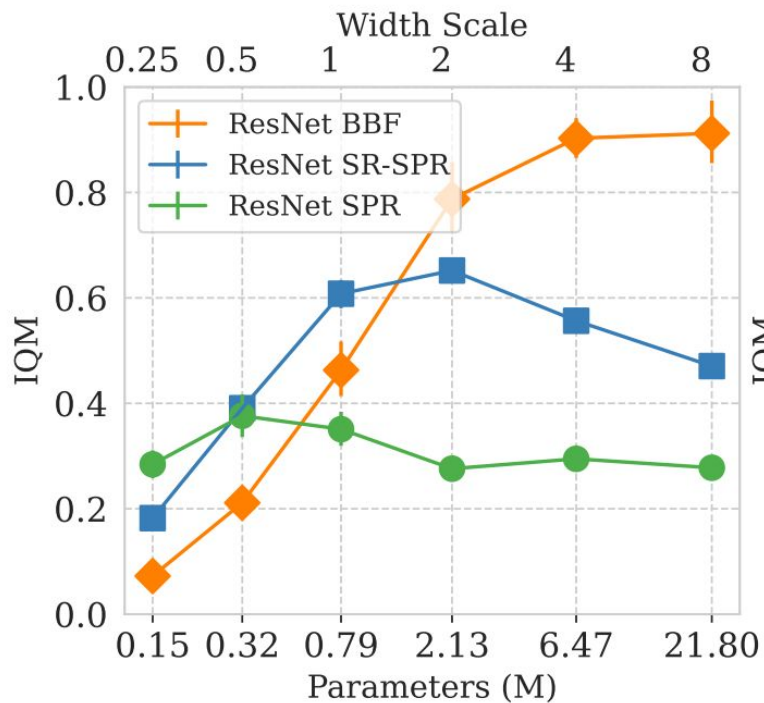
Bigger, Better, Faster: Human-level Atari with human-level efficiency



Motivation

- How to scale RL networks?
 - Simply increasing weights don't work

모델을 *scaling*을 어떻게 해야 할까?





Methodology

- Base agent: SR-SPR with $RR=8$
- Harder reset: Convolution layer reset 20% -> 50%
- Larger network
 - Impala-CNN (15-layer ResNet)
 - Width 4x (8x better, but 4x to reduce computation)
- Receding update horizon:
 - $n=10 \rightarrow 3$ bootstrap vs. $n=3$ for rainbow
- Increase discount factor:
 - $\gamma: 0.97 \rightarrow 0.997$
- Weight decay:
 - AdamW with decay value of 0.1
- Remove Noisy net:
 - No increase in performance
 - Noisy net adds noise to weights == duplicate copies of finally two layers
 - Removal : increase flops by 2.5x and reduce memory by 1.6x

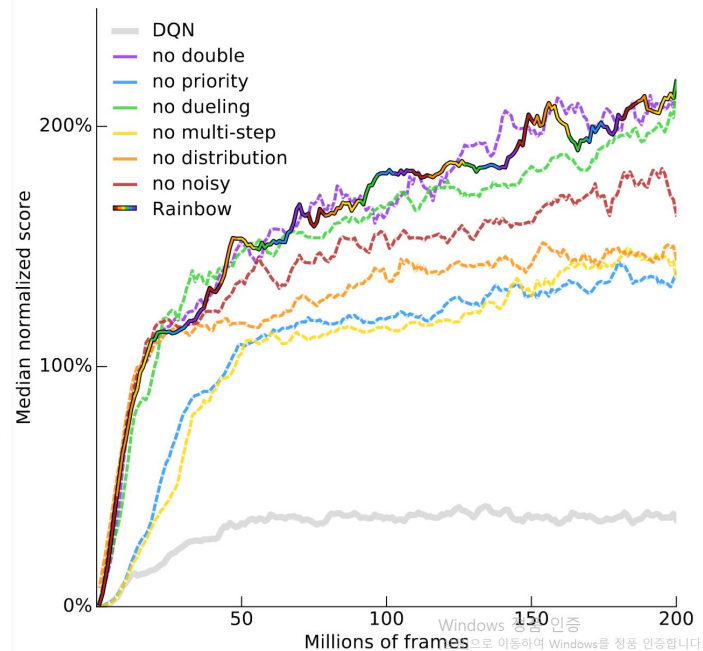


Findings

- Self supervision: SPR (self-supervision) objective benefits from larger networks
- Target network: Important component in replay ratio scaling
- Scaling is not enough
- Computational efficiency:
 - EfficientZero: 512 CPU, 4GPU in 8.5 hours
 - IRIS: Half of A100 for 1 week
 - SR-SPR: 1/4 of A100, 1 CPU for 24 hours
 - BBF: 1/2 of A100, 1 CPU for 10 hours.

Revisiting Atari 100k benchmark

- Isn't it solved and too easy?
 - Only uses 26/55 games
 - Still has optimality gap
- Relatively hard with small computational need
- New metric:
 - Rainbow's final performance with 2 hours of gameplay?
- Data scaling
 - Higher steps (1M steps to compare with Rainbow's 20M steps)
 - Lower steps (20k, 50k)





Implementation Details

- SPR
 - Rainbow DQN
 - Encoder
 - Transition model
 - projection
- SR-SPR
 - Replay Ratio
 - Parameter Rest
- BBF
 - Impala CNN (15-layer ResNet)
 - Horizon update
 - Gamma exponential scheduler
 - Weight decay scheduler



Additional Papers

- Rainbow DQN